If you remember from the chapter on logic, the == operator has a higher precedence than the bitwise operators, so it is evaluated first. On the final line of the example above, we're testing the result of `$leastSignificant ^ $leastSignificant`. Because of the parentheses, the line above evaluates `$leastSignificant == true` before using the result of that in the bitwise XOR operation.

Finally, here's one more logical bitwise operator, which will also serve to illustrate the memory inefficiency of PHP for handling integer data at the bit level:

```
$num = bindec('11111111');
var_dump(decbin(~ $num));
var_dump(~ $num);
```

```
Output:
string(64)
"1111111111111111111111111111111111111111111111111111111100000000"
int(-256)
```

Here the operator is ~, which is (logically enough… sorry) the NOT operator. This flips every bit to its opposite value, so *1001* would become *0110* and so on. Except that in this case, we've ended up with rather a lot of ones on the left-hand side and a decimal value of *-256* instead of *0*. Why?

Well, remember at the beginning of the section it was stated that all integers are a C data type called *zend_long*? When we specified that we wanted an integer with the first eight bits set to *1* (giving a value of *0xFF* or decimal *255*), what PHP actually assigned in memory was this: *00000000 00000000 00000000 00000000 00000000 00000000 00000000 11111111*. The bitwise NOT of that 64-bit value is, of course, *11111111 11111111 11111111 11111111 11111111 11111111 11111111 00000000*, which is *-256* (you'll see why below).

Constants

There are several built-in PHP constants that provide easy access to commonly used mathematical constants such as *Pi* or *e*:

```
$r = 10;

var_dump(M_PI * $r ** 2);
```

```
Output:
float(314.1592653589793)
```